NOPOS CIRCUITS EXPERIMENTS 1 - REVISED VERSION

FIRST CIRCUIT:

Task: Odd idxs sum up until random position.

SECOND CIRCUIT:

Task:

Given a vector of 64 tokens selected *w/o* replacement (every token appears only once) predict the absolute position regardless on anything.

Config setup:

```
d_model=128,
d_head=128,
n_heads=1,
d_mlp=512,
```

Notebook name (from which I took the plots):

`analyze_predict_abs_pos_15_07_24_2024-07-18 00:58:55.342699.ipynb`

More refined notebook:

`analyze_predict_abs_pos_15_07_24_2024-07-18 00:58:55.342699-For Yoav.ipynb`

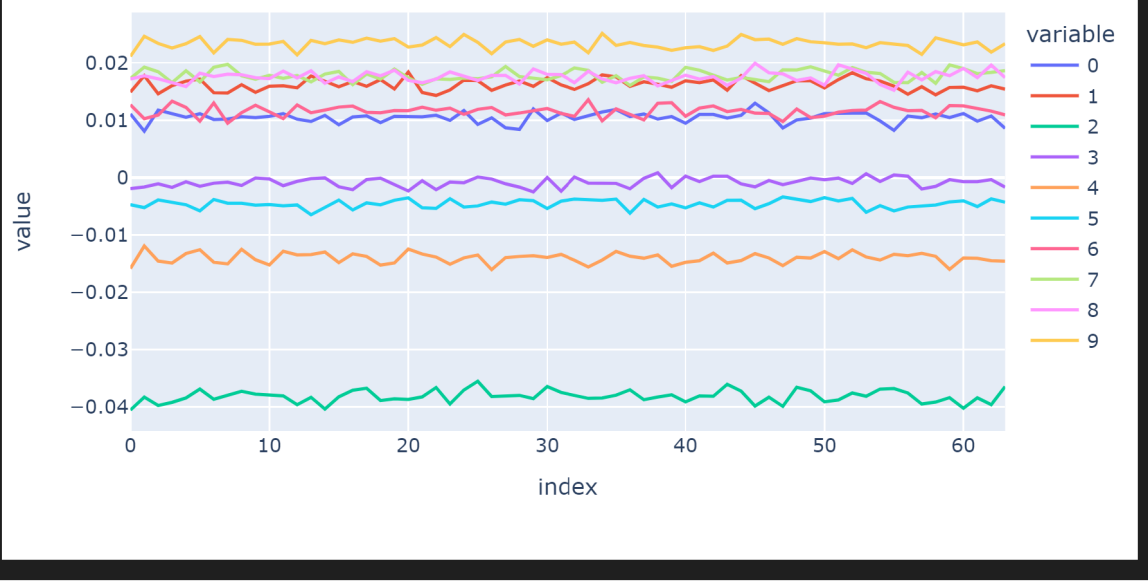
Model path (saved to my downloads dir):

`/home/nlp/matan_avitan/git/nopos_locating/notebooks/predict_abs_pos/predict_abs_pos_15_07_24_2024-07-18 00:58:55.342699/epoch=680-step=85125.ckpt`

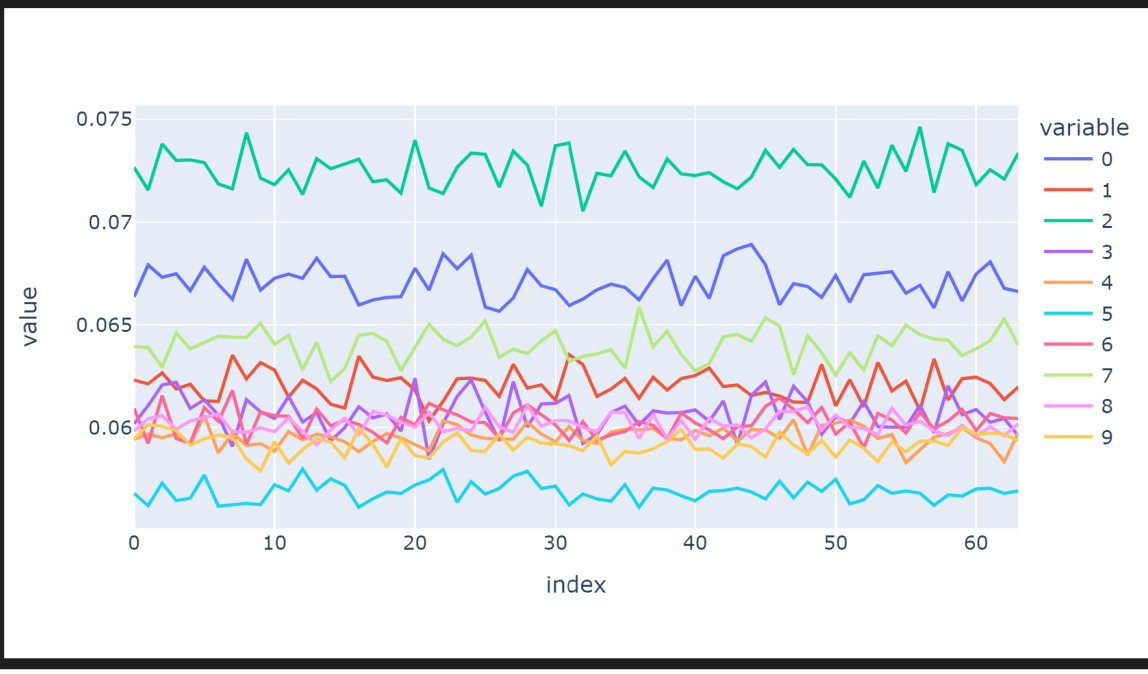
תקציר האבחנות שלי על המעגל:

1. ב-resid_pre של הבלוק הראשון, אם נסתכל על הממוצע של הדוגמאות וניקח צ'אנלים רנדומלים נראה כי אין שום יכולת לרשת לקודד מיקום עם הייצוג הקיים של ה-resid_pre:

```
line(cache['blocks.0.hook_resid_pre'].squeeze(2).mean(0)[:,20:30])
```

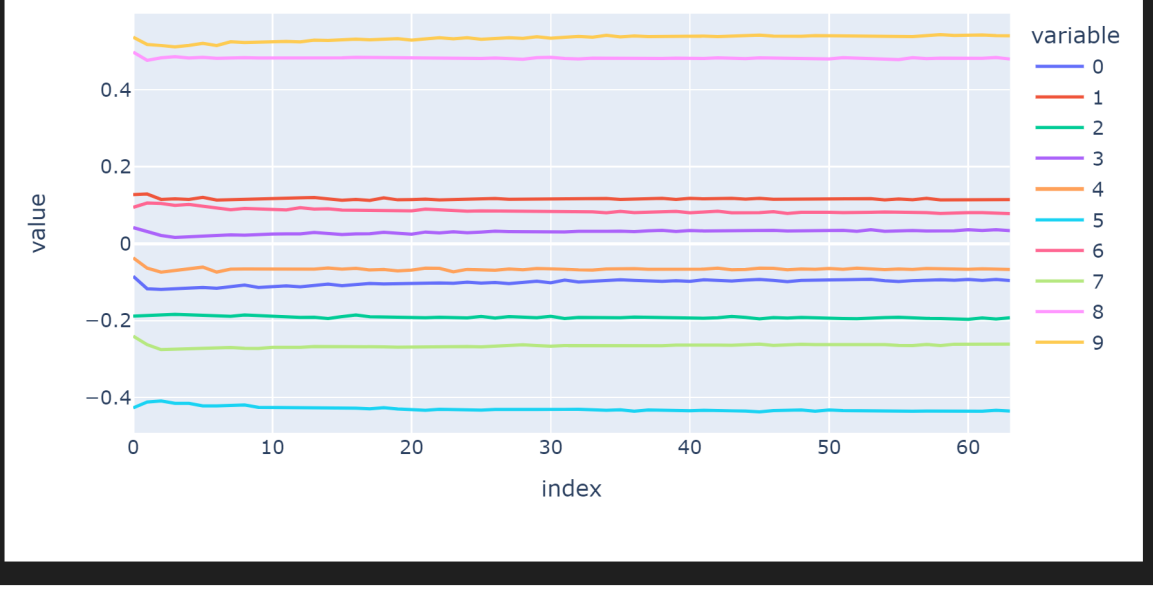



```
line(cache['blocks.0.hook_resid_pre'].squeeze(2).std(0)[:,20:30])
```

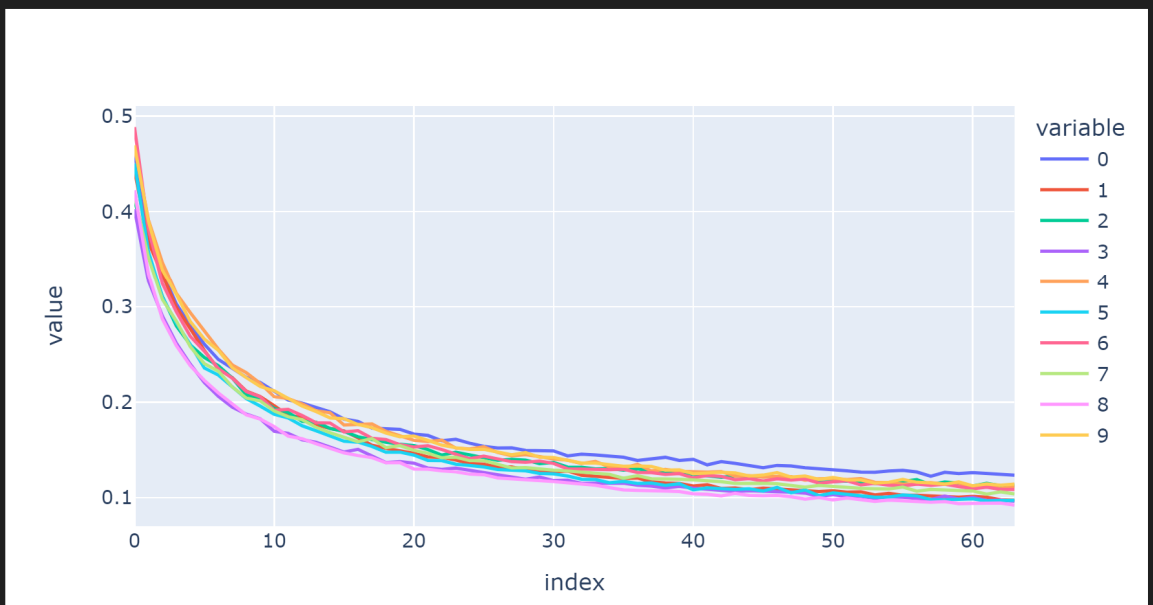


2. החלק המעניין מגיע עכשיו, אם נסתכל על הייצוג אחרי הראש אטנשיין הראשון נראה שאין שום אינדיקציה בתוחלת אבל יש בשונות!

```
line(cache['blocks.0.hook_resid_mid'].squeeze(2).mean(0)[:,20:30])
```

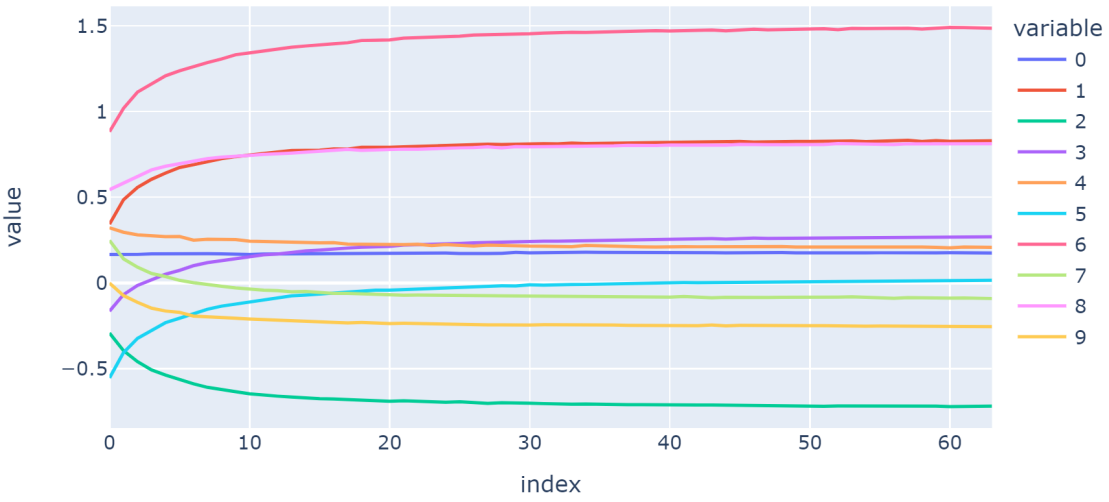



```
line(cache['blocks.0.hook_resid_mid'].squeeze(2).std(0)[:,20:30])
```



3. לאחר מכן ה-MLP יודע לקחת את הייצוג של המיקום מהשונות ולקודד אותו בתוחלת:

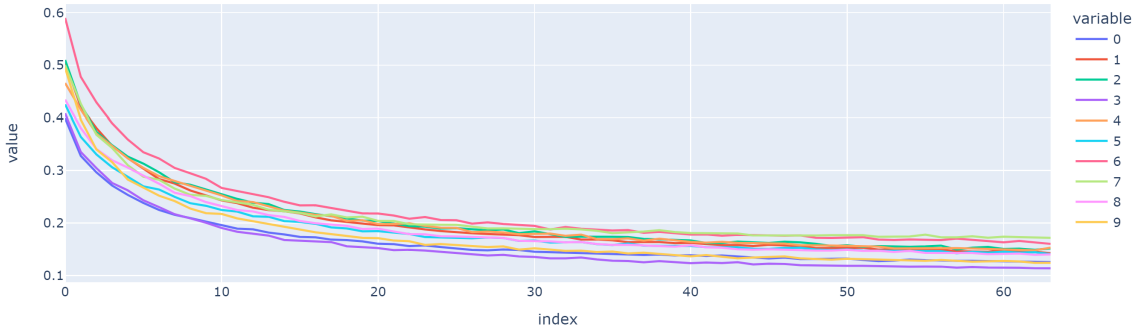
```
line(cache['blocks.0.hook_resid_post'].squeeze(2).mean(0)[:,20:30])
```



(אפשר לראות שהמונוטוניות בשונות נשארת גם אחרי ה-MLP אבל זה פחות קריטי לסיפור)

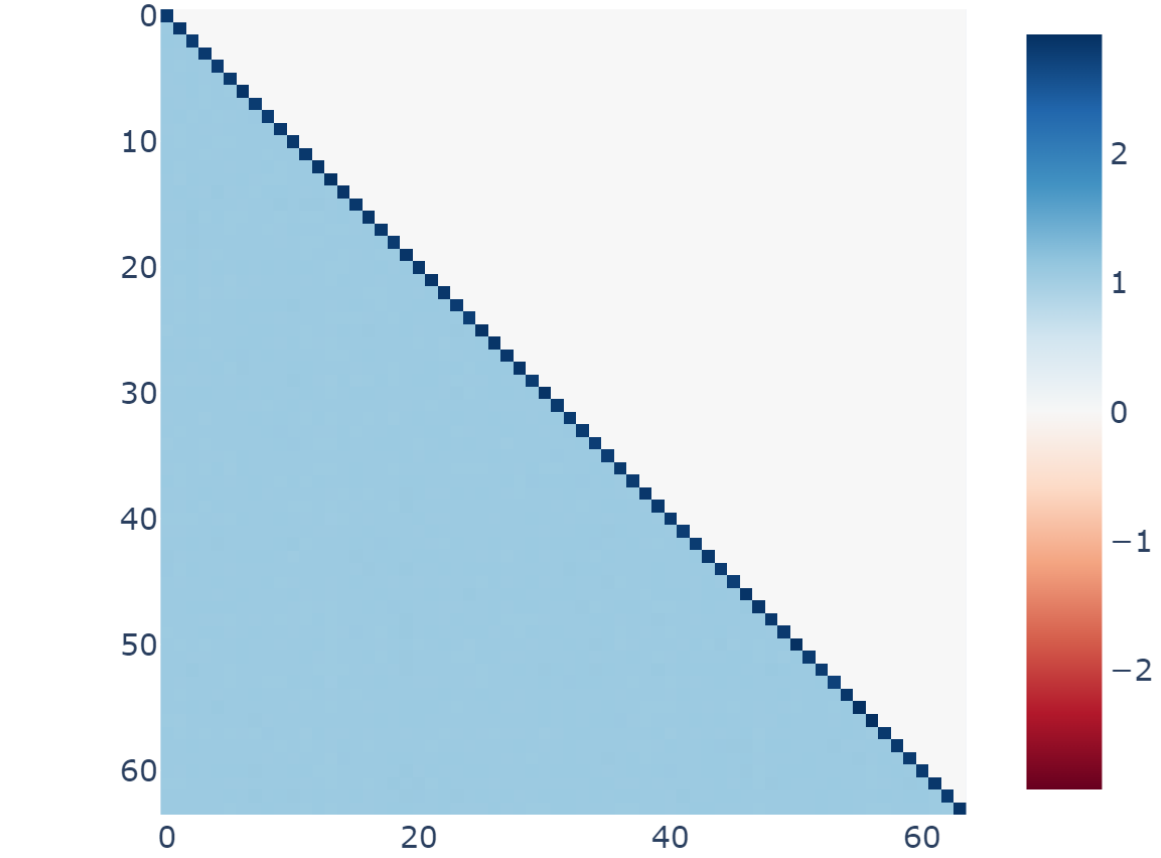
```
line(cache['blocks.0.hook_resid_post'].squeeze(2).std(0)[:,20:30])
```

0.0s Python

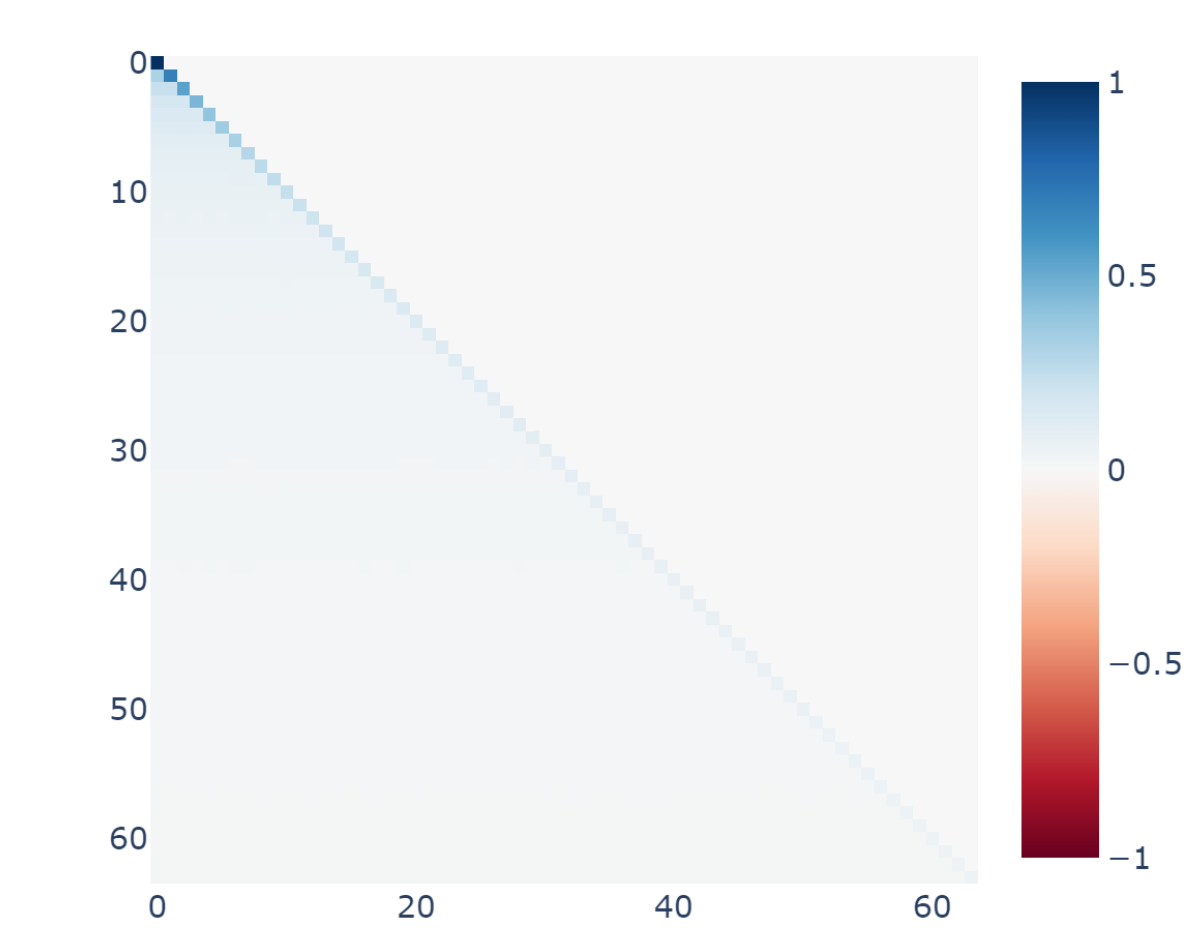


4. עכשיו בוא נדבר על איך הקידוד הזה בשונות של ה-resid_mid של הבלוק הראשון קורית:

באופן עקבי הראש הראשון מתכנס לדפוס הנל בשכבה הראשונה, כאשר i=j יש ערך אטנשיין מסוים וכאשר i שונה j יש ערך אחר (וקטן יותר.)

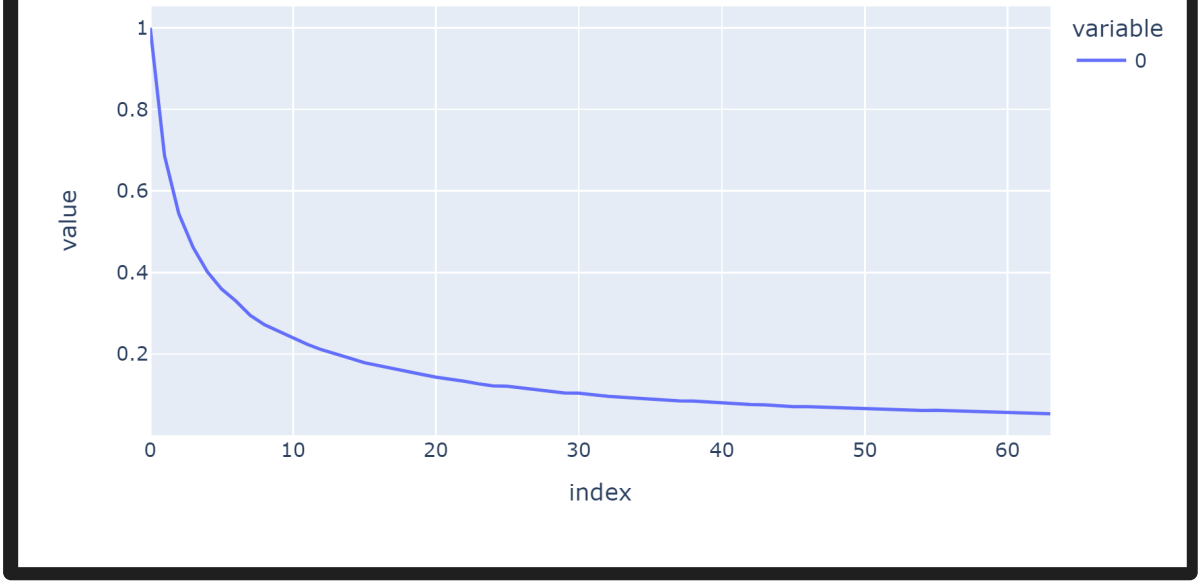


5. מאחר ואנחנו כל פעם מוסיפים איבר אחד לאיברים שאנחנו עושים להם attention אנחנו מקבלים שהאלכסון דועך מונוטונית:



6. ניתן להתבונן באלכסון מקרוב, יש לציין שאופי הדעיכה האקספוננציאלי נובע מה-softmax והוא רכיב במעגל שלנו:

```
line(torch.diagonal(cache["attn", 0][:, [i], :, :].mean([0, 1])))
```

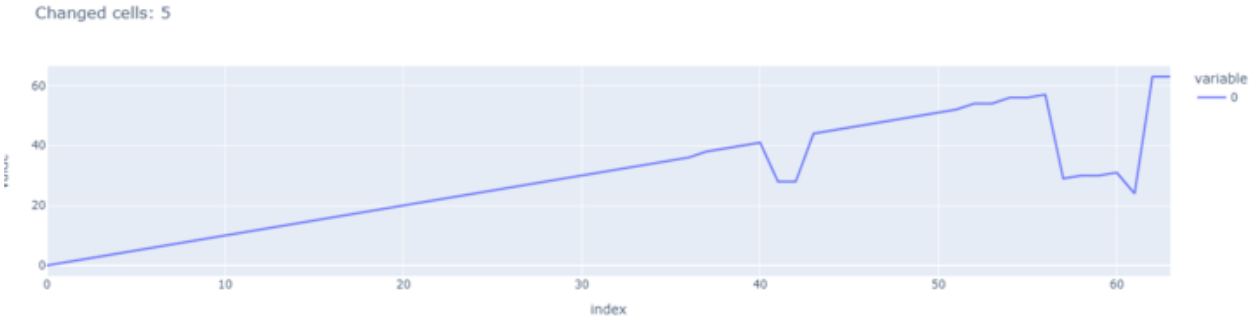


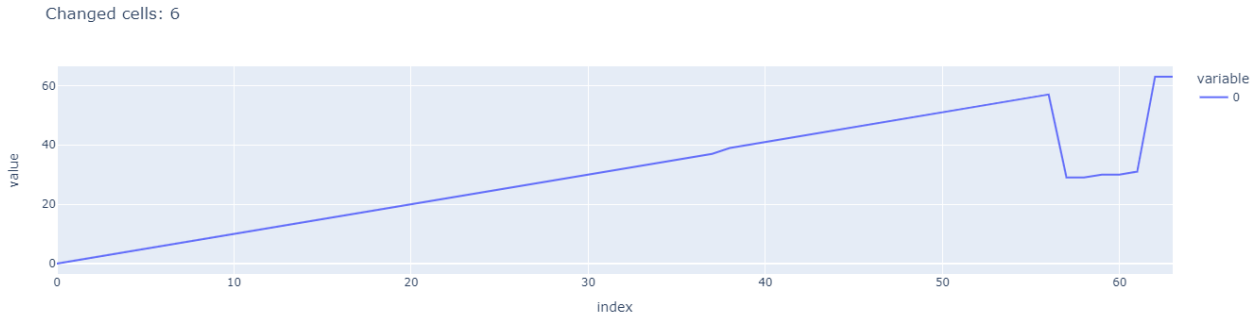
7. מה שמעניין זה שהממוצע אחרי ה-softmax זהה עבור כל המיקומים (השורות):

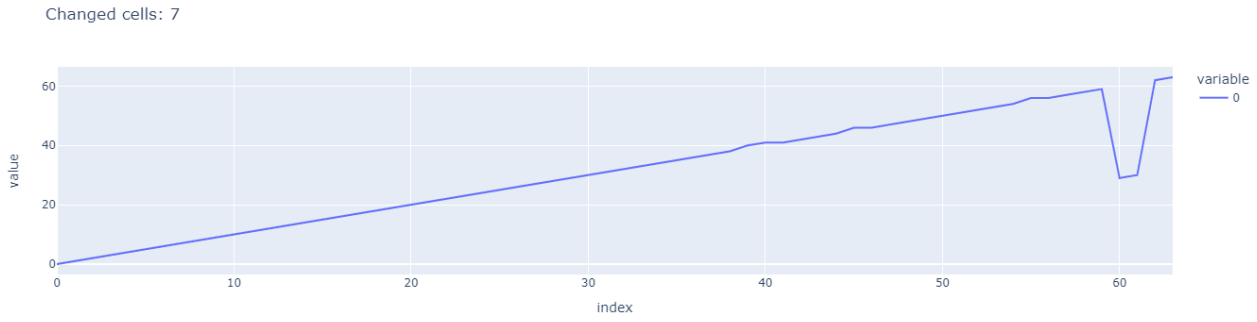
tensor([0.0156, 0.0156, 0.0156,

לקחתי וקטור של 64 ערכים קבועים וביקשתי ממנו לחזות את המיקום והוא כשל וחזה רק את המיקום 0 בכולם.

לאחר מכן שיניתי רק את האיבר הראשון לאיבר רנדומלי כלשהו ואז הוא הצליח קצת יותר, שיניתי גם את השני לאיבר רנדומלי והוא הצליח עוד וכן הלאה, מאז שהוא הגיע ל-9 distinct tokens הוא הצליח לחזות את ה-absolute position באופן מושלם!





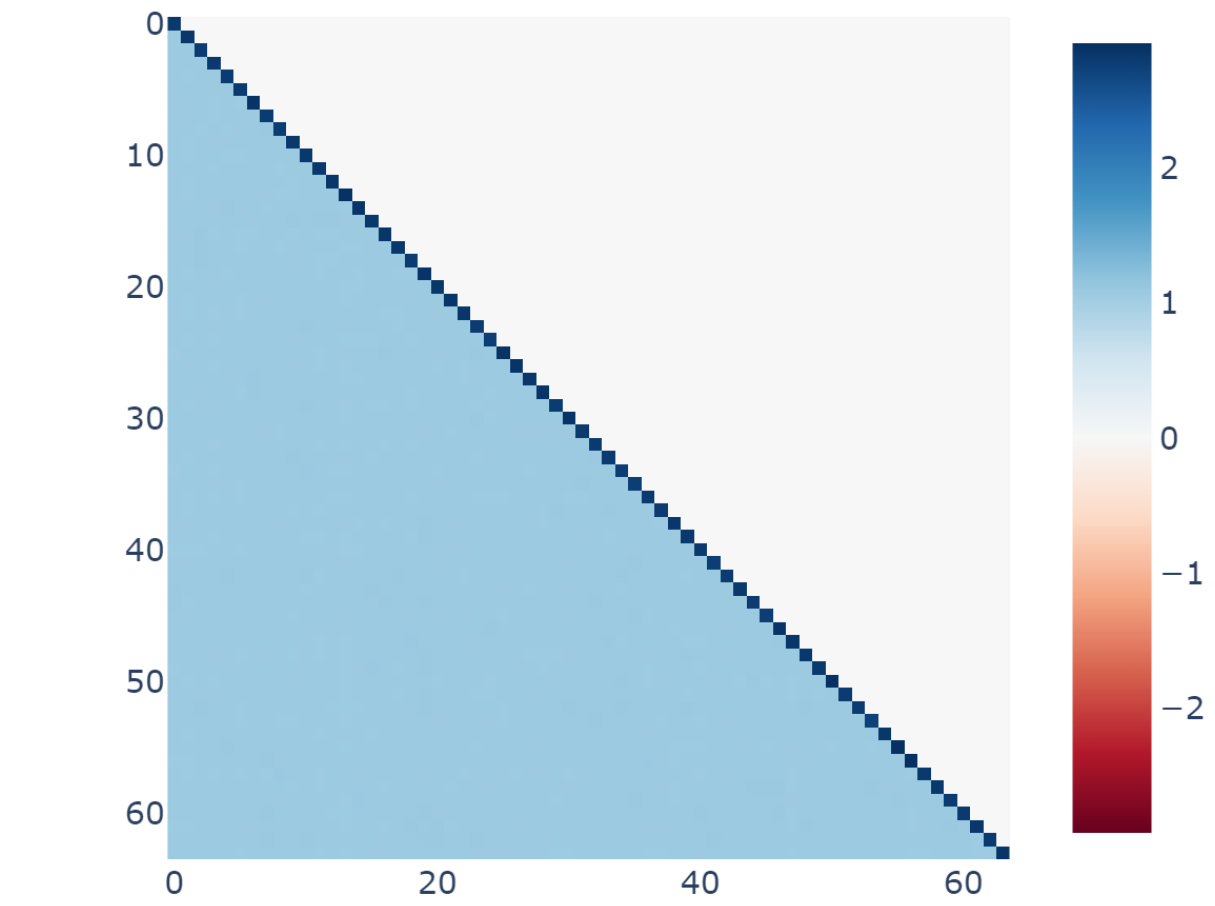


מצאתי מעגל נוסף שבאמצעותו הרשת מקודדת מיקום והוא ממש מעניין.

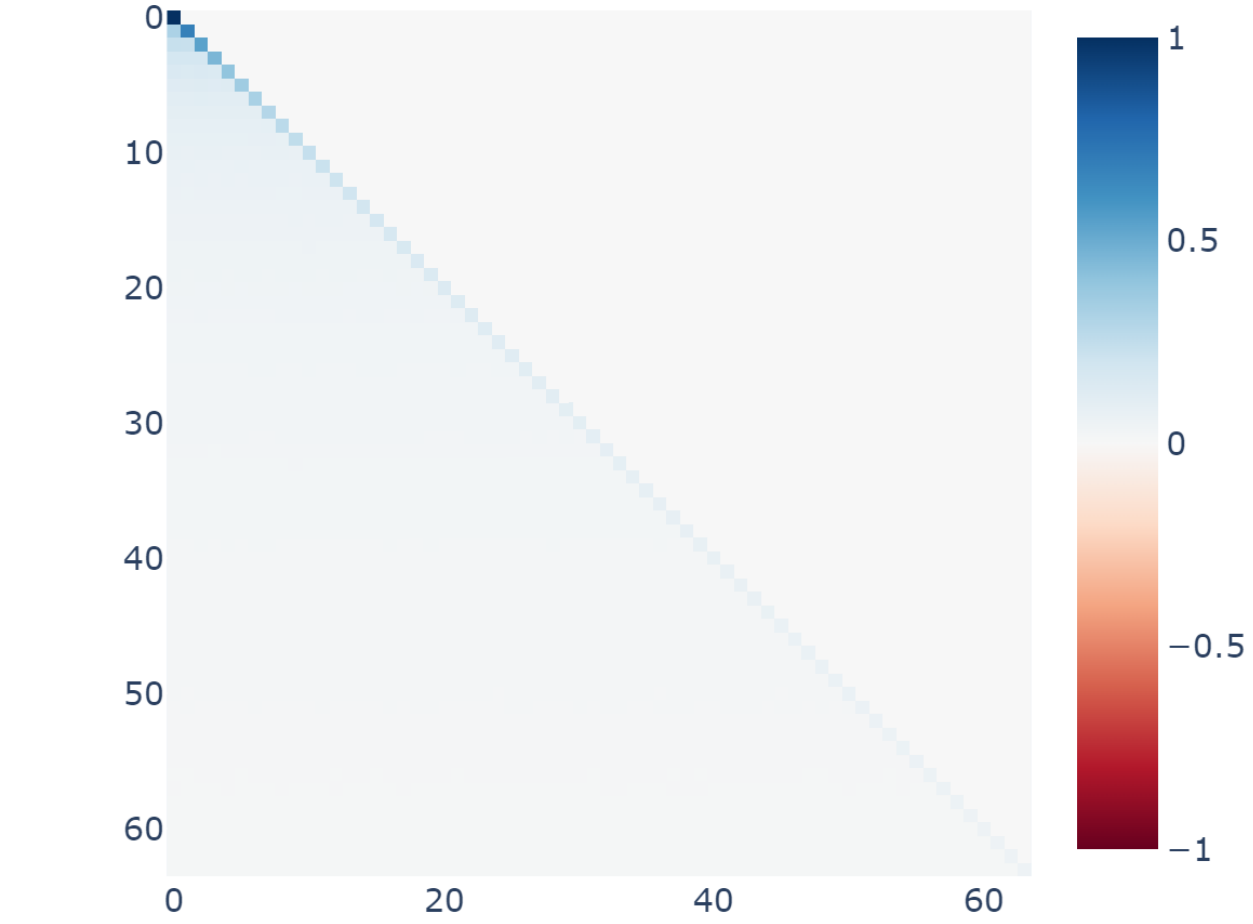
הרשת מקודדת את המיקום בשונות של ה-layer normalization.

איך היא עושה את זה?

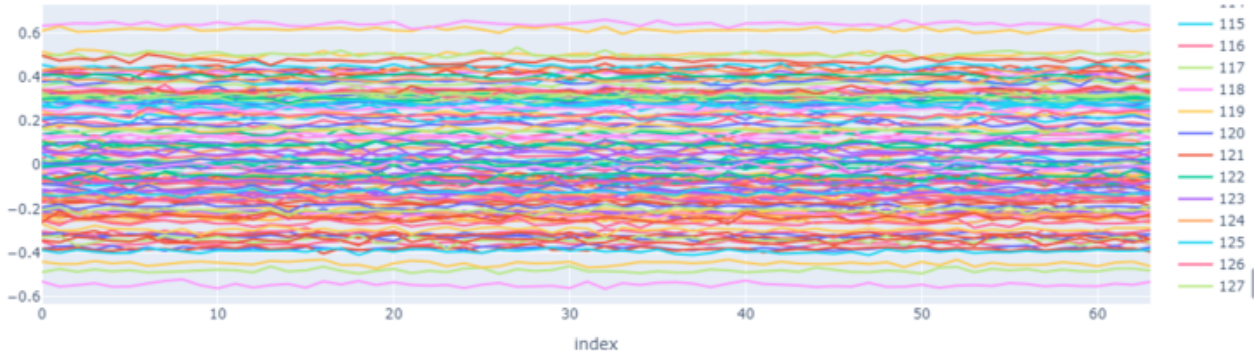
כאן הרשת לא מכנסת את הראש אטנשיין הראשון למשהו אחיד לחלוטין אלא, למעט האלכסון כל שאר האיברים מתפלגים אחיד



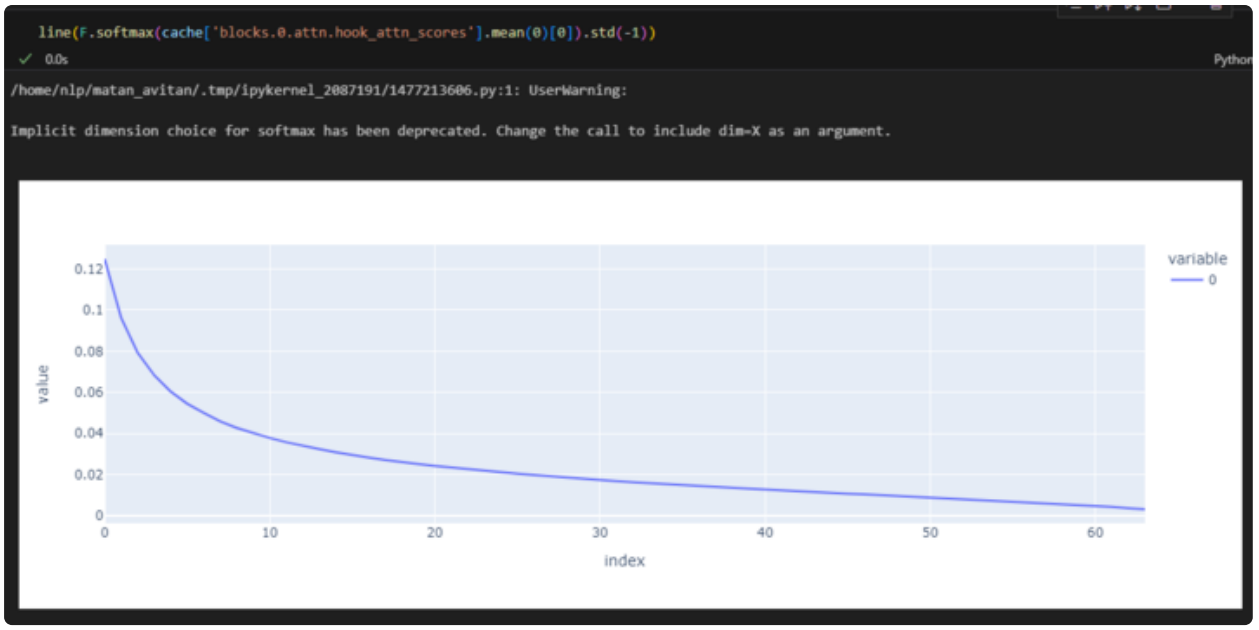
מאחר וכל פעם אנחנו מוסיפים איבר אחד (השווה לכל הקודמים) ה-softmax של האלכסון דועך מונוטונית



במקרה של המעגל הנוכחי, אין ספייק במטריצת אמבדינג אלא היא פונקצייה קבועה כפונקציה של המיקום, ולכן גם הוקטורי values הם פונקציה קבועה עבור כל צ'אנל כפי שניתן לראות כאן:



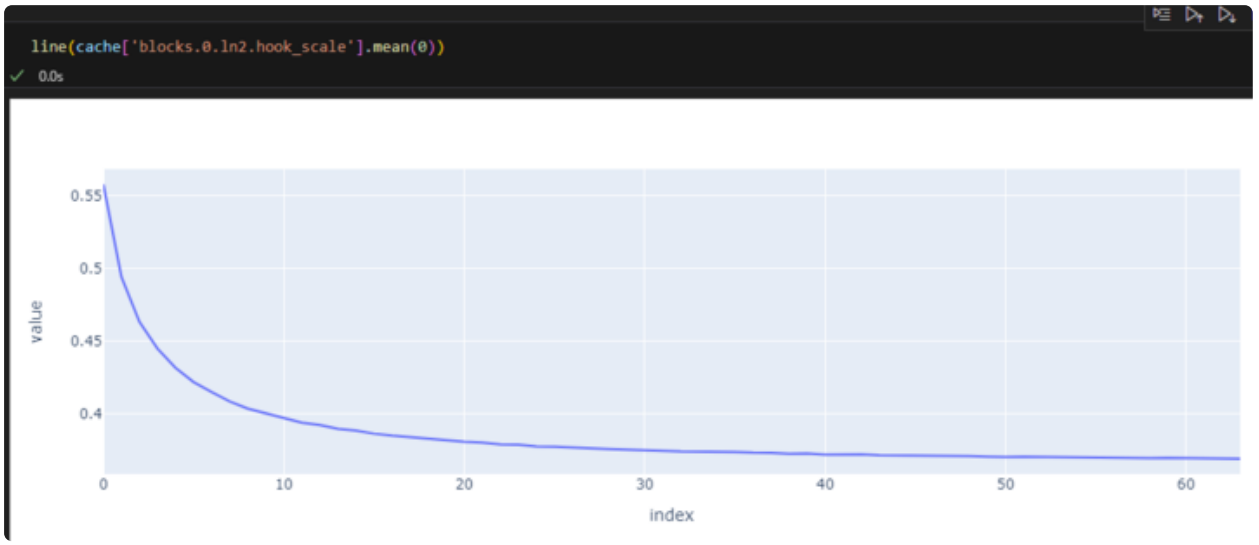
עכשיו מגיע הקסם, אם נתבונן בסטיית תקן של הראש אטנשיין הראשון נקבל את הפונקציה הבאה:



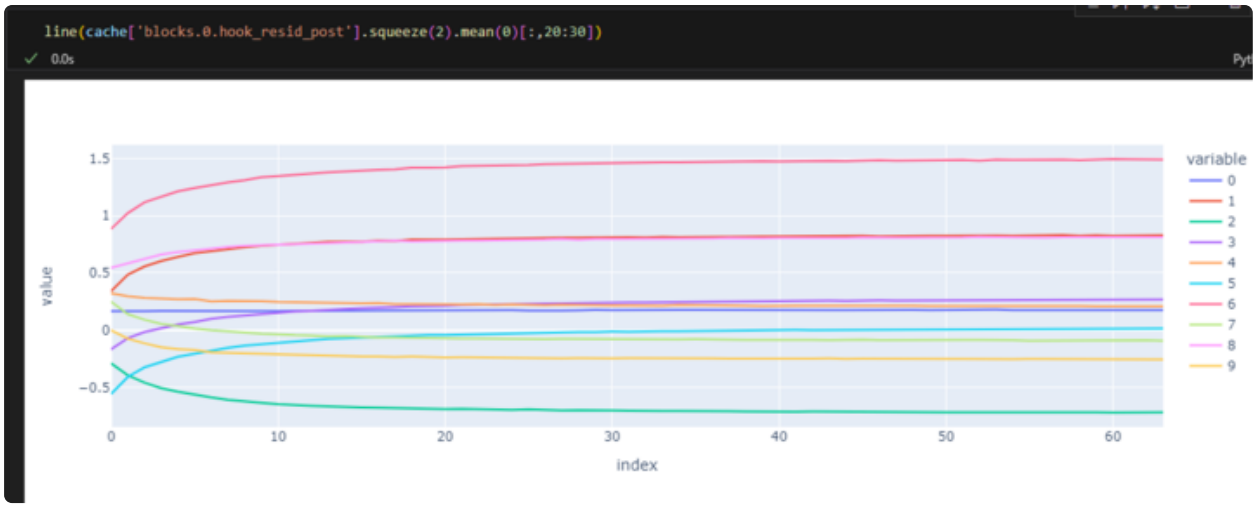
מאחר והשלב הבא הוא layer normalization, הרשת שומרת את הסטיות תקן האלה ומנרמלת את הוקטורי values הקבועים.

משהו קבוע חלקי משהו מונוטוני שווה משהו מונוטוני.

כאן ניתן לראות את הסטיות תקן שנשמרות פר צ'אנל ב-layer normalization:



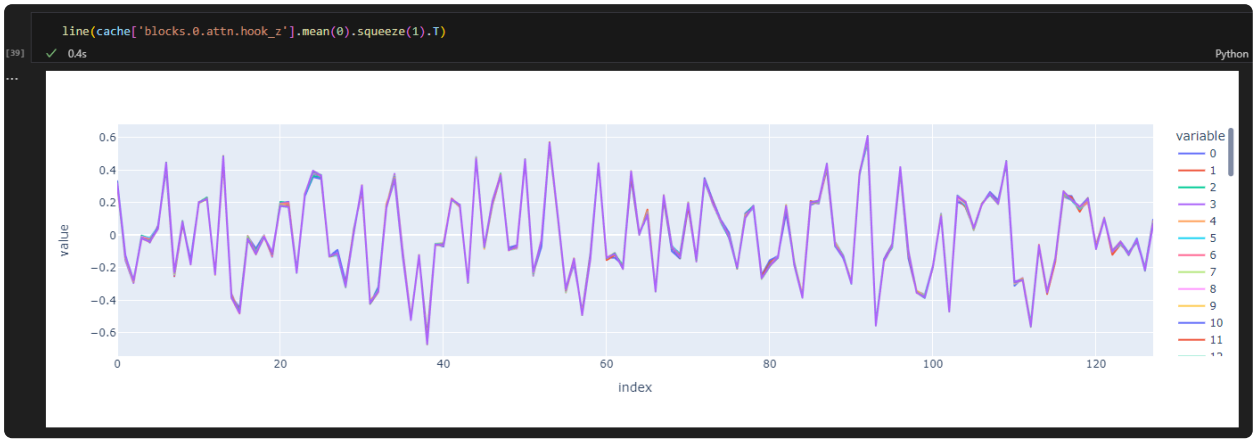
ולכן בשלב הבא נקבל המון צ'אנלים מונוטונים כפונקציה של המיקום ב-residual stream:

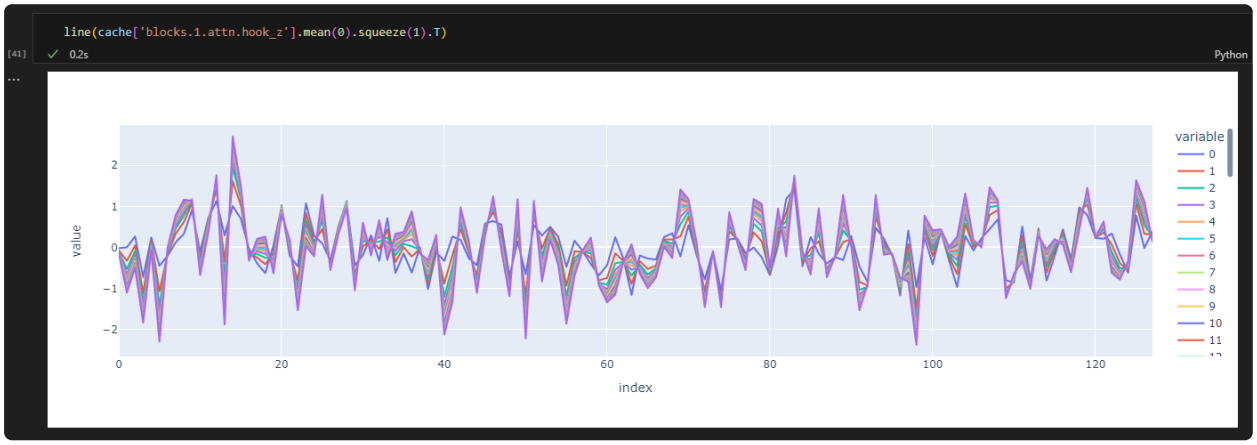


בקיצור בנוסף למעגל הקודם מצאתי מעגל נוסף, שמה שמגניב בו זה שהוא מקודד את המיקום באמצעות הסטיית תקן שמחושבת ב-layer normalization שמגיעה אחרי ה-self attention הראשון!

הבנחה מעניינת זה שה-layer normalization הוא הרכיב היחיד שיכול לערבב בין הצ'אנלים בצורה כזו שמחשבת את השונות ביניהם באופן מפורש.

מבט שונה על העיניין שבו המידע מקודד בשונות של המטריצת attention score הראשונה היא איך שנראים הצ'אנלים כפונקציה של המיקומים, בוא נראה איך נראים הוקטורי Z אחרי ה-attention matrix הראשונה אל מול השנייה:





רואים איך פתאום פר מיקום יש שונות גדולה בין הצ'אנלים השונים.

משהו מוזר שהבחנתי בו זה זה שהאלכסון של המטריצת אטנשיין מתכנס לפאי בערך:

```
array([3.1444147, 3.0899024, 3.1546335, 3.1616037, 3.12696 ,
3.142754 , 3.1414561, 3.1311152, 3.1101122, 3.148159 ,
3.1425302, 3.1342614, 3.138472 , 3.1534839, 3.1584976,
3.1573687, 3.1648772, 3.1575181, 3.1251593, 3.1500804,
3.1585555, 3.1385481, 3.131962 , 3.1168551, 3.1369565,
3.1486807, 3.1538777, 3.146658 , 3.142858 , 3.1397781,
3.1241436, 3.1422198, 3.1487458, 3.1545844, 3.1319294,
3.1685443, 3.1238754, 3.1465476, 3.136578 , 3.1314185,
3.1392136, 3.1634312, 3.1285837, 3.1158254, 3.0909224,
3.1007314, 3.1315494, 3.1499314, 3.1445103, 3.1401513,
3.1484816, 3.1226537, 3.1674552, 3.149239 , 3.1328366,
3.1373808, 3.1200821, 3.125422 , 3.1339762, 3.1380558,
3.138939 , 3.1202753, 3.1375759, 3.136274 ], dtype=float32)
```

Yields the best intervention results (layer is: blocks.0.ln2.hook_scale)

```
value[:, idx1, :] = torch.tensor(act_idx2.mean(0))
```

logits, loss = model.run_with_hooks(inputs_s, return_type='both', fwd_hooks=[(f'blocks.1.attn.hook_z', swap_norm_hook)])

Worked great